

Assignment 10: StockSync — Warehouse Inventory

Pending



Objective

Build a **FastAPI-based Warehouse Inventory API** to manage items, warehouses, stock levels, low-stock alerts, and inventory valuation.

The assignment should demonstrate:

- `OAuth2PasswordRequestForm`
- Role-Based Access Control
- Response Model Filtering
- Stock Management
- Query Parameter Filtering
- Alembic Migrations
- Validation and Error Handling

Technical Requirements

- **Framework:** FastAPI
- **Authentication:** OAuth2 using `OAuth2PasswordRequestForm`
- Roles:
 - admin
 - manager
- Use **Alembic** for database schema migrations
- `unit_cost` must not be exposed to non-admin users
- Stock quantity must never go below 0

Data Model

Data Models

User

- id
- email
- hashed_password
- role

Item

- id
- name
- unit_cost
- selling_price
- quantity

Warehouse

- id
- location_name

Inventory

- item_id
- warehouse_id
- quantity

API Routes

POST /items

Add a new item to the inventory.

Required fields:

- Name

- Unit cost
- Selling price
- Quantity

Validation:

- `selling_price >= unit_cost`
 - Quantity must be greater than or equal to 0
-

GET /items/{id}

Return details of a specific item.

Response model rules:

- **Admin:** Can view `unit_cost`
- **Manager:** Must not see `unit_cost`

Use separate response models or response filtering to ensure internal cost information is hidden from non-admin users.

Return 404 Not Found if the item does not exist.

PATCH /items/{id}/stock

Increase or decrease the stock quantity of an item.

Example:

Current Stock: 20

Change: -5

Updated Stock: 15

Rules:

- Stock must never become negative.
 - Return an appropriate error if the requested decrease would result in stock below 0.
 - Return 404 Not Found if the item does not exist.
-

GET /inventory/low-stock

Return items where the quantity is below a specified threshold.

Default:

`threshold = 10`

Example:

`/inventory/low-stock?threshold=5`

The threshold must be configurable using a query parameter.

GET /inventory/valuation**Admin Only**

Calculate the total value of the current inventory.

Use:

`Inventory Value = unit_cost × quantity`

The response should provide the total stock valuation.

Managers must not be allowed to access this endpoint.

OAuth2 Login

Use:

`OAuth2PasswordRequestForm`

for user login.

Users should authenticate using their credentials and receive an access token.

Protected APIs should validate the authenticated user's role.

Response Model Filtering

Create suitable response schemas so that sensitive information is hidden where required.

For example:

Admin Response

id
name
unit_cost
selling_price
quantity

Manager Response

id
name
selling_price
quantity

unit_cost must never be exposed to non-admin users.

Alembic Migrations

Use **Alembic** for database schema versioning.

Students must use migrations when creating or updating database tables.

The project should contain the required Alembic configuration and migration files.

Validation & Business Rules

- `selling_price >= unit_cost`
 - Stock must never be negative.
 - Low-stock threshold must be configurable.
 - Default low-stock threshold must be 10.
 - Only admins can access inventory valuation.
 - `unit_cost` must not be visible to managers.
 - Alembic migrations must be used for schema changes.
-

Error Handling

Handle appropriate errors for:

- Item not found
- Invalid login
- Unauthorized access
- Negative stock
- Invalid price values
- Selling price lower than unit cost
- Invalid threshold
- Invalid request data

Use appropriate HTTP status codes such as 400, 401, 403, 404, and 422.

API Testing

All APIs should be testable through FastAPI Swagger:

/docs

Test:

- OAuth2 login
- Item creation
- Price validation
- Admin and manager responses
- Stock increase/decrease
- Negative stock prevention
- Low-stock filtering
- Inventory valuation
- Role-based access

• Atomic migrations

Submission Instructions

Submit the **complete project as ONE ZIP file through the LMS.**

The ZIP file should include:

- Complete source code
- Alembic configuration and migration files
- requirements.txt
- README.md
- All files required to run the project

ZIP File Name

StudentName_StockSync.zip

Example:

RahulKumar_StockSync.zip

Important

- Submit only **one ZIP file**.
 - Do not upload individual files separately.
 - Do not include `venv`, `.venv`, `__pycache__`, or unnecessary folders.
 - Test the application before submission.
 - Ensure the correct ZIP file is uploaded successfully to the LMS.
 - Submit before the deadline mentioned in the LMS.
-

Evaluation Criteria

The assignment will be evaluated based on:

- Correct API functionality

- OAuth2 authentication
- Role-Based Access Control
- Response model filtering
- Stock update logic
- Negative stock prevention
- Low-stock filtering
- Inventory valuation
- Price validation
- Alembic migration implementation
- Error handling
- Code quality and organization
- Successful project execution
- Proper ZIP submission

Expected Outcome

By completing this assignment, you should be able to build a structured **FastAPI warehouse inventory system using OAuth2 authentication, response model filtering, stock validation, role-based access, inventory calculations, and**

Upload Assignment